

Contents

Phase 1: Business Definition, Questions & KPIs	2
1. Business Overview.....	2
2. Business Questions	3
3. Key Performance Indicators	3
Phase 2 & 3: Database Design (8 Normalized Tables - 3NF)	4
Key Constraints Needed.....	5
Phase 4 & 5: Implementation Roadmap	5
Local Development (PostgreSQL or SQL Server).....	5
Local_Queries_Success.png	6
Query:1.....	7
Query:2.....	7
AWS RDS Deployment	8
.....	8
Phase 6: Cloud Data Pipeline (S3 + Glue + Athena)	9
Created IAM Role: PulseChargeGlueRole	10
Step 1: Create a Glue Database.....	10
Database Catalog	10
Step 2: Choose data sources and classifiers	11
Step 3: Configure security settings	11
Step 4: Set output and scheduling.....	12
Amazon Athena	15
Data Transformation & Pipeline Automation (Glue ETL).	16
Step-by-Step Setup: CSV to Parquet Pipeline	16
Phase 7: Data Mining & Analytics	19
Queries on Athena	19
Query 1: Overall Network Performance (Aggregation)	19
Query 2: High-Volume Customer Segmentation (Thresholding Analysis)	20
Query 3: Session Energy Categorization (Classification Logic)SQL	21

Query 4: Customer Usage Tier Ranking	23
Phase 8: Dashboard & Final Submission Checklist	26
Conclusion:	26
PulseCharge Data Lake & Analytics Pipeline	26

Phase 1: Business Definition, Questions & KPIs

1. Business Overview

- **Company:** PulseCharge Network North America

- **Business Model:** PulseCharge operates fast-charging EV stations. Revenues are generated via energy delivery fees (kWh), session usage tariffs, and monthly subscriptions.
- **Core Problem:** Dynamic demand leads to station bottlenecks during peak hours, under-utilized chargers in suburban areas, unexpected downtime, and customer churn. PulseCharge needs a centralized database and data pipeline to optimize station availability, dynamic tariffs, and proactive maintenance.

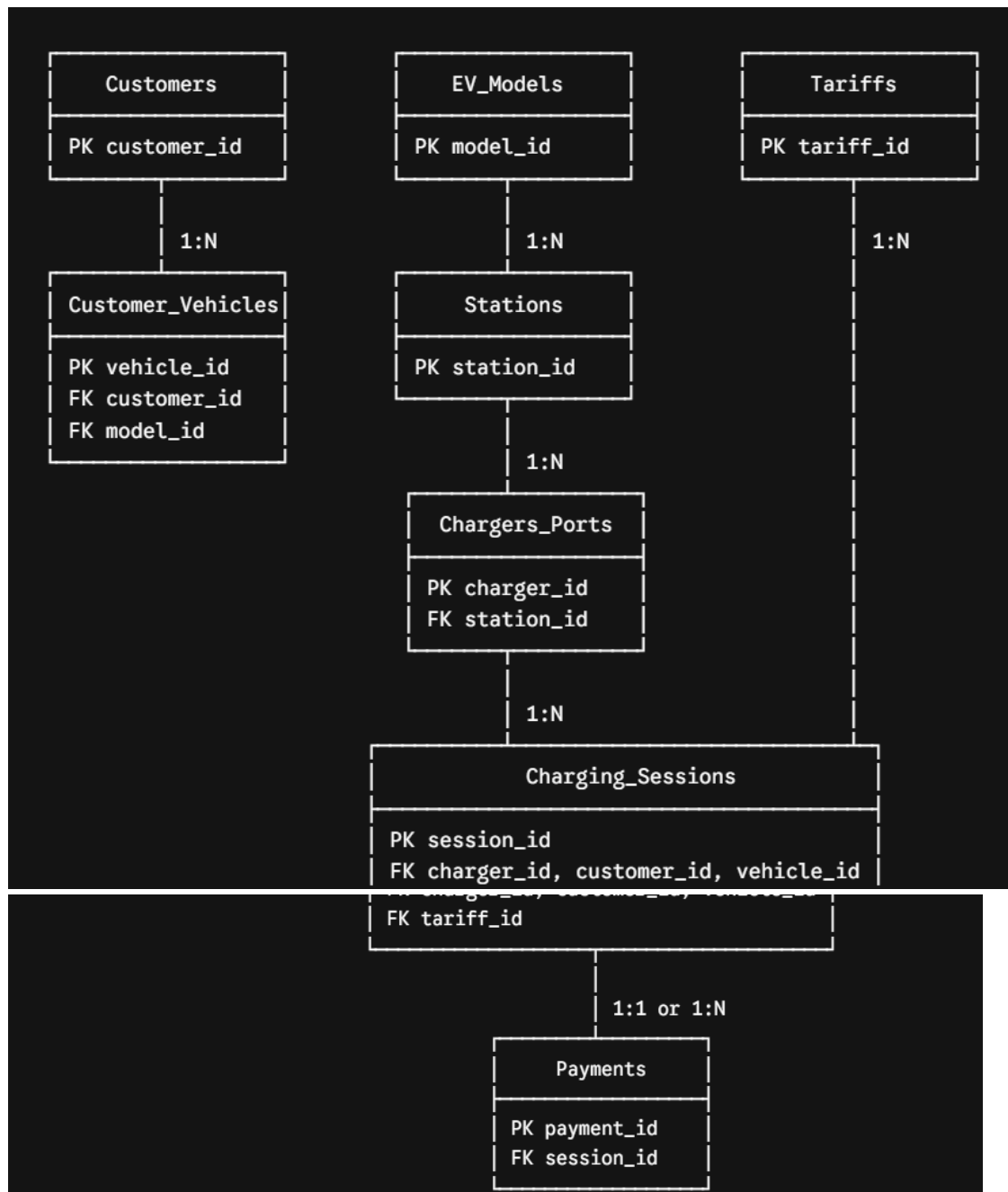
2. Business Questions

1. Which charging stations generate the highest revenue per month?
2. What is the overall station occupancy rate during morning/evening peak hours versus off-peak hours?
3. Which charger types (Level 2 vs. DC Fast Chargers) experience the highest downtime rate?
4. What is the average session duration and energy delivered (kWh) per EV model type?
5. How does dynamic pricing (tariffs) impact customer session length and total revenue?
6. Which customer segments exhibit the highest churn or lowest retention rate over 90 days?
7. What are the most frequent root causes listed in unresolved maintenance tickets?
8. Which geographic clusters of stations represent high-demand vs. low-utilization corridors?

3. Key Performance Indicators

- Average Session Revenue (\$/session): $(\text{Total Revenue}) / (\text{Total Charging Sessions})$
- Station Occupancy Rate (%): $(\text{Total Charging Hours}) / (\text{Total Available Charger Hours}) \times 100$
- Peak Energy Delivered (kWh): Sum of energy dispensed during peak window (07:00–10:00 & 16:00–19:00).
- Charger Downtime Rate (%): $(\text{Total Outage Hours} / \text{Total Operating Hours}) \times 100$
- 90-Day Customer Retention Rate (%): $(\text{Active Repeat Customers in 90 days} / \text{Total Customers}) \times 100$

Phase 2 & 3: Database Design (8 Normalized Tables - 3NF)



Key Constraints Needed

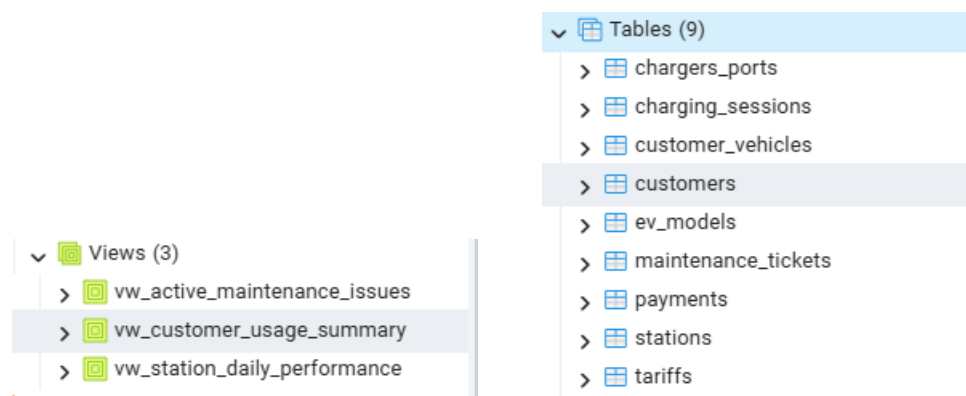
- NOT NULL on session start times, customer emails, rates.
- UNIQUE constraint on customer.email and chargers_ports.serial_number
- CHECK constraints: energy_kwh ≥ 0 , port_status IN ('Available', 'Occupied', 'Offline', 'Maintenance'), downtime hours ≥ 0
- We have prepared Python Data Generator Script (generate_ev_data.py).

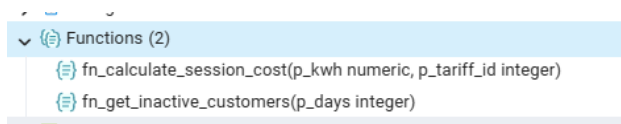
Phase 4 & 5: Implementation Roadmap

Local Development (PostgreSQL or SQL Server)

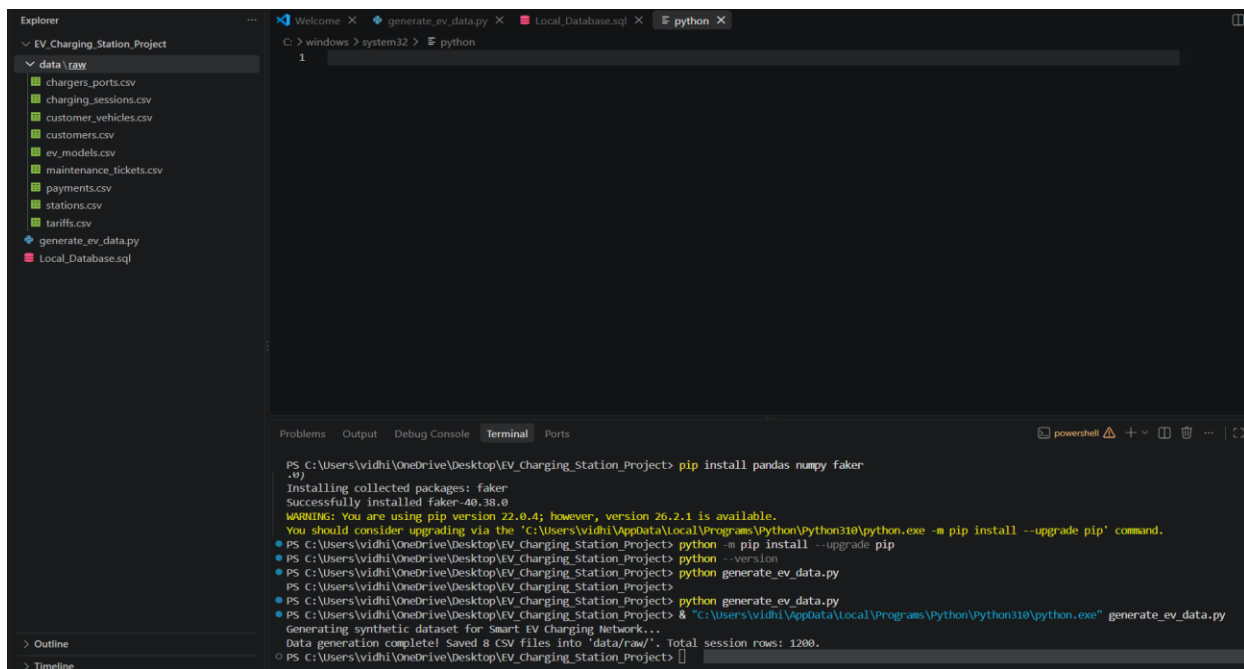
1. Generate synthetic dataset (1,000+ rows across all tables) using Python
2. Write Local_Database.sql to build local schema, tables, foreign keys, and constraints.
3. Import data into local database using CSV Import Wizard or Python pandas
4. Create required database objects:
 - 3 Views: e.g., vw_station_daily_performance, vw_active_maintenance_issues, vw_customer_usage_summary
 - 3 Stored Procedures/Functions: e.g., sp_calculate_session_cost(), sp_flag_offline_chargers(), sp_customer_churn_risk()
 - 12 Analytical Queries: CTES, Window functions (AVG() OVER(), RANK()), Aggregates, Multi-table Joins.

We have SQL schema script (Local_Database.sql) formatted for PostgreSQL. It builds all 8 normalized tables in 3NF with strict data integrity constraints, primary keys, and foreign key relationships.





1. The Local_Database.sql script open in VS Code or pgAdmin containing the 12 query statements



Local_Queries_Success.png

- The Query Tool in pgAdmin displaying a successful execution result grid for one of the key analytical queries.

Query:1

Query

Query History

```
1  -- Q1: Total Revenue and Energy Delivered by Station Category
2  SELECT s.category, COUNT(cs.session_id) AS total_sessions, ROUND(SUM(cs.total_
3  FROM Stations s
4  JOIN Chargers_Ports cp ON s.station_id = cp.station_id
5  JOIN Charging_Sessions cs ON cp.charger_id = cs.charger_id
6  GROUP BY s.category ORDER BY revenue DESC;
7
```

Data Output

Messages

Notifications

+

📄

▼

📋

▼

🗑️

📦

📥

📈

SQL

Showing rows: 1 to 4

✎

📄

 Page No: 1 of 1

⏪

⏴

⏵

⏩

	category character varying (30)	total_sessions bigint	revenue numeric
1	Suburban Mall	381	6832.14
2	Residential Transit	319	5661.94
3	Highway Corridor	285	5023.03
4	Urban Hub	215	4264.36

Query:2

🔗

pulsecharge_db/postgres@PostgreSQL 18

▼

📦

📁

▼

📄

▼

✎

▼

🔍

▼

No limit

▼

📄

▶

📢

▼

E

📊

▼

📄

Query

Query History

```
7
8  -- Q2: Top 5 Station Locations by Occupancy Volume
9  SELECT s.station_name, COUNT(cs.session_id) AS session_count
10 FROM Stations s
11 JOIN Chargers_Ports cp ON s.station_id = cp.station_id
12 JOIN Charging_Sessions cs ON cp.charger_id = cs.charger_id
13 GROUP BY s.station_name ORDER BY session_count DESC LIMIT 5;
```

Data Output

Messages

Notifications

+

📄

▼

📋

▼

🗑️

📦

📥

📈

SQL

Showing rows: 1 to 5

✎

📄

 Page No: 1 of 1

⏪

⏴

⏵

⏩

	station_name character varying (100)	session_count bigint
1	PulseCharge - Simmons Spur	117
2	PulseCharge - Carly Motorway	79
3	PulseCharge - Courtney Loop	79
4	PulseCharge - Kendra Canyon	75
5	PulseCharge - Jenny Motorw...	70

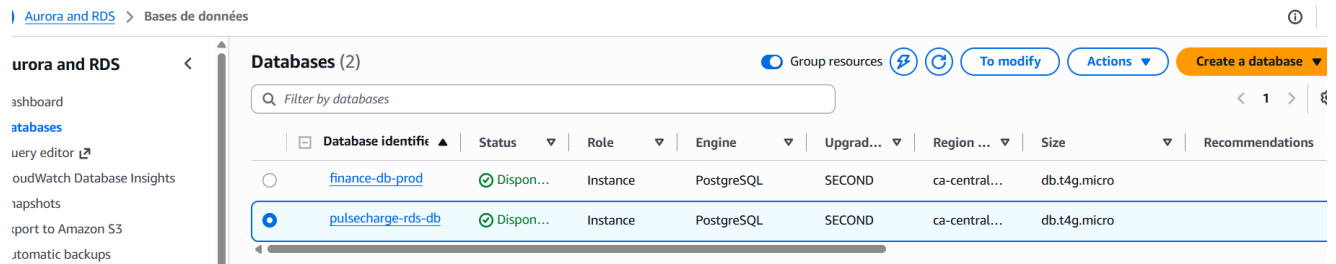
AWS RDS Deployment

- 1.Provision RDS PostgreSQL or RDS SQL Server (db.t3.micro free-tier eligible)
 - 2.Set Security Group: restrict port (e.g., 5432 for Postgres) to My IP (YOUR_LOCAL_IP/32)
 - 3.Run AWS_Deployment.sql script via pgAdmin/SSMS connected to the RDS endpoint.
- VPC, Subnets, Availability Zones are created.

The screenshot displays the AWS Management Console interface for an Internet Gateway. The breadcrumb navigation shows 'VPC > Internet gateways > igw-0e34c3fea47843b06'. A green notification banner at the top states: 'Internet gateway igw-0e34c3fea47843b06 successfully attached to vpc-05b36778c051be999'. The main heading is 'igw-0e34c3fea47843b06 / pulsecharge-igw'. Below this, the 'Details' section shows the Internet gateway ID as 'igw-0e34c3fea47843b06', the State as 'Attached', the VPC ID as 'vpc-05b36778c051be999', and the Owner as '385697366557'. The 'Tags (1)' section shows a single tag with the key 'Name' and value 'pulsecharge-igw'.

☐	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
☐	-	rtb-005e9bba9057d4045	-	-	Yes	vpc-033881134eeb46aaf Spe
☐	-	rtb-03e26982e80ef116a	-	-	Yes	vpc-05b36778c051be999
☐	-	rtb-0f2ac91127c1ddd7a	-	-	Yes	vpc-066c0186bd5eb0004 pul
☐	pulsecharge-rtb-private1-ca-central-1a	rtb-09eb4ceaa41f1d165	-	-	No	vpc-066c0186bd5eb0004 pul
☐	pulsecharge-rtb-private2-ca-central-1b	rtb-0b2a6b538117abc8b	-	-	No	vpc-066c0186bd5eb0004 pul
☐	pulsecharge-rtb-public	rtb-0b19d03aa5fd1bd6d	4 subnets	-	No	vpc-066c0186bd5eb0004 pul

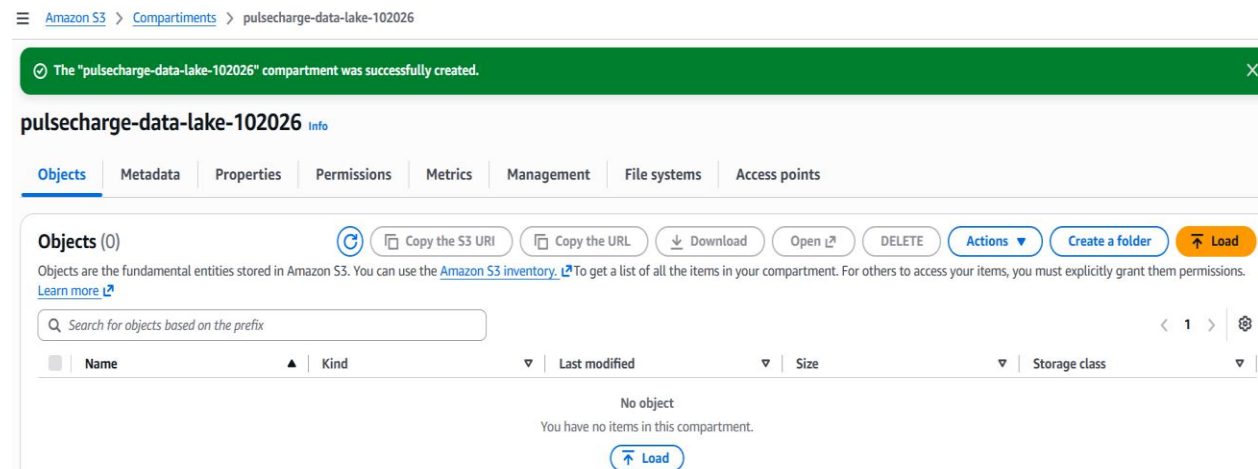
On AWS RDS, database “**pulsecharge-rds-db**” is created.



Phase 6: Cloud Data Pipeline (S3 + Glue + Athena)

1. S3 Bucket: Setup structure:
 - s3://pulsecharge-analytics-data/raw/ (Original CSV exports)
 - s3://pulsecharge-analytics-data/cleaned/ (Post-ETL / Normalized CSVs)
 - s3://pulsecharge-analytics-data/athena-results/
2. AWS Glue Crawlers: Run Glue Crawler targeting s3://.../cleaned/ to populate the Glue Data Catalog database schema.

S3 Datalake: pulsecharge-data-lake-102026



Created folder raw-data/

Amazon S3 > Compartments > pulsecharge-data-lake-102026 > raw-data/

raw-data/ Copy the S3 URI

Objects (9) Copy the S3 URI Copy the URL Download Open DELETE Actions Create a folder Load

Objects are the fundamental entities stored in Amazon S3. You can use the [Amazon S3 Inventory](#) to get a list of all the items in your compartment. For others to access your items, you must explicitly grant them permissions. [Learn more](#)

Search for objects based on the prefix

☐	Name	Kind	Last modified	Size	Storage class
☐	chargers_ports.csv	csv	10 Sep 2026 11:26:26 AM EDT	4.6 KB	Standard
☐	charging_sessions.csv	csv	10 Sep 2026 11:26:25 AM EDT	85.0 KB	Standard
☐	customer_vehicles.csv	csv	10 Sep 2026 11:26:25 AM EDT	14.4 KB	Standard
☐	customers.csv	csv	10 Sep 2026 11:26:25 AM EDT	24.3 KB	Standard
☐	ev_models.csv	csv	10 Sep 2026 11:26:25 AM EDT	368.0 o	Standard
☐	maintenance_tickets.csv	csv	10 Sep 2026 11:26:25 AM EDT	4.6 KB	Standard
☐	payments.csv	csv	10 Sep 2026 11:26:25 AM EDT	66.3 KB	Standard
☐	stations.csv	csv	10 Sep 2026 11:26:24 AM EDT	2.9 KB	Standard
☐	tariffs.csv	csv	10 Sep 2026 11:26:24 AM EDT	164.0 o	Standard

Created IAM Role: PulseChargeGlueRole

☐	Name	Service	Created
☐	PulseChargeGlueRole	Service AWS: glue	-

Step 1: Create a Glue Database

In the AWS Console, type AWS Glue in the top search bar.

In the left navigation menu under Data Catalog, click Databases.

Click Add database: Name: **pulsecharge_catalog_db**

Click Create database.

Database Catalog

AWS Glue > Databases

AWS Glue

- Getting started
- SageMaker Unified Studio
- ETL jobs
 - Visual ETL
 - Notebooks
 - Job run monitoring
- Data Catalog tables
- Data connections
- Workflows (orchestration)
- Zero-ETL integrations
- Data Catalog**
 - Catalogs
 - Databases**
 - Tables
 - Stream schema registries
 - Schemas
 - Connections
 - Crawlers

Your catalog, more capabilities Set up

Discover, govern, and trust your data in SageMaker Unified Studio with AI-powered exploration, built-in quality scores, and secure sharing across your organization

Databases (2) Last updated (UTC) September 10, 2026 at 15:32:28 Edit Delete Actions Add database

A database is a set of associated table definitions, organized into a logical group. Choose catalog

385697366557

Filter databases

☐	Name	Description	Location URI	Source catalog	Created on (UTC)
☐	default	-	-	385697366557	June 10, 2026 at 00:56:22
☐	pulsecharge_catalog_db	-	-	385697366557	September 10, 2026 at 15:32:30

Chose S3 as data sources

Step 2: Choose data sources and classifiers

Click Add a data source.

Data source: Select S3.

Location of S3 data: Select “In this account”.

S3 path: Click Browse S3 and select your raw-data/ folder inside **pulsecharge-data-lake-102026 (or type s3://pulsecharge-data-lake-102026/raw-data/)**.

Click Add an S3 data source.

Click Next.

The screenshot shows a multi-step wizard interface. On the left, a vertical list of steps is shown: Step 1 (Set crawler properties), Step 2 (Choose data sources and classifiers - currently active), Step 3 (Configure security settings), Step 4 (Set output and scheduling), and Step 5 (Review and create). The main content area is titled 'Choose data sources and classifiers'. It contains a 'Data source configuration' section with two radio buttons: 'Not yet' (selected) and 'Yes'. Below this is a table of 'Data sources (1)' with columns for Type, Data source, and Parameters. The table lists one source: S3, with the path s3://pulsecharge-data-lake-102026 and the parameter Recrawl all. At the bottom, there is a section for 'Custom classifiers - optional' with a description. Navigation buttons at the bottom right include Cancel, Previous, and Next.

Step 1 Set crawler properties

Step 2 **Choose data sources and classifiers**

Step 3 Configure security settings

Step 4 Set output and scheduling

Step 5 Review and create

Choose data sources and classifiers

Data source configuration

Is your data already mapped to Glue tables?

☒ Not yet
Select one or more data sources to be crawled.

☐ Yes
Select existing tables from your Glue Data Catalog.

Data sources (1) [Q help](#)

The list of data sources to be scanned by the crawler.

Type	Data source	Parameters
☐ S3	s3://pulsecharge-data-lake-102026	Recrawl all

Custom classifiers - optional

A classifier checks whether a given file is in a format the crawler can handle. If it is, the classifier creates a schema in the form of a StructType object that matches that data format.

[Cancel](#) [Previous](#) [Next](#)

Choose IAM GlueRole

Step 3: Configure security settings

Existing IAM role: Select PulseChargeGlueRole.

Click Next.

Step 1

Set crawler properties

Step 2

Choose data sources and classifiers

Step 3

Configure security settings

Step 4

Set output and scheduling

Step 5

Review and create

Configure security settings

IAM role [Q help](#)

IAM permissions

To access other AWS services and resources you need an IAM role. Choose a role with the right permissions.

Create default role

Use another role

A new role will be created with the necessary permissions.

Role name

PulseChargeGlueRole

Maximum 64 characters. Use alphanumeric and +,=, @, _ characters.

Lake Formation configuration - optional

Allow the crawler to use Lake Formation credentials for crawling the data source. [Learn more.](#)

Use Lake Formation credentials for crawling S3 data source

Checking this box will allow the crawler to use Lake Formation credentials for crawling the data source. If the data source is registered in another account, you must provide the registered account ID. Otherwise, the crawler will crawl only those data sources associated to the account. Only applicable to S3, Glue Catalog, Iceberg, and Hudi data sources.

Security configuration - optional

Enable at-rest encryption with a security configuration.

Cancel

Previous

Next

Step 4: Set output and scheduling

Target database: Select pulsecharge_catalog_db (created in Step 1).

Frequency: Keep as “On demand”.

Click Next.

Review the configuration and click Create crawler.

Step 1

Set crawler properties

Step 2

Choose data sources and classifiers

Step 3

Configure security settings

Step 4

Set output and scheduling

Step 5

Review and create

Review and create

Step 1: Set crawler properties

Edit

Set crawler properties

Name	Description	Tags
pulsecharge-data-lake-crawler	-	-

Step 2: Choose data sources and classifiers

Edit

Data sources (1) [Q help](#)

The list of data sources to be scanned by the crawler.

Type	Data source	Parameters
S3	s3://pulsecharge-data-lake-102026	Recrawl all

Step 3: Configure security settings

Edit

Configure security settings

IAM role	Security configuration	Lake Formation configuration
PulseChargeGlueRole	-	-

Step 4: Set output and scheduling

Edit

Set output and scheduling

Database	Table prefix - optional	Maximum table threshold - optional	Schedule
pulsecharge_catalog_db	-	-	On demand

pulsecharge-data-lake-crawler

pulsecharge-data-lake-crawler

One crawler successfully created
The following crawler is now created: "pulsecharge-data-lake-crawler"

pulsecharge-data-lake-crawler Last updated (UTC)
September 10, 2026 at 15:42:24 [Run crawler](#) [Edit](#) [Delete](#)

Crawler properties			
Name	IAM role	Database	State
pulsecharge-data-lake-crawler	PulseChargeGlueRole	pulsecharge_catalog_db	READY
Description	Security configuration	Lake Formation configuration	Table prefix
-	-	-	-
Maximum table threshold			
-			

► Advanced settings

Now that crawler is created, let's run it to discover the schema of your operational CSVs and create tables in your Glue Data Catalog.

- Step 1: Run the Crawler in the AWS Glue Console, select Crawlers from the left navigation pane under Data Catalog. Check the box next to pulsecharge-data-lake-crawler. Click "Run" button at the top right.
- Step 2: Monitor Crawler Status. Watch the Status column for pulsecharge-data-lake-crawler. It will transition through:
Running → Glue is scanning your S3 bucket folders.

Stopping → Glue is writing the discovered schemas to your Catalog.

Ready → The scan is complete.

The entire process usually takes about 1 to 2 minutes

- Step 3: Verify Discovered Tables
 - In the left navigation menu under Data Catalog, click Tables.
 - Change the Database filter dropdown to **pulsecharge_catalog_db**.

We can see separate tables listed for each of your subfolders (e.g., sessions, stations, users).

Click on any table name to open its details and verify that AWS Glue correctly detected column names and data types (such as string, bigint, or double).

Crawler successfully starting

The following crawler is now starting: "pulsecharge-data-lake-crawler"

Notifications002000

Crawlers

A crawler connects to a data store, progresses through a prioritized list of classifiers to determine the schema for your data, and then creates metadata tables in your data catalog.

Crawlers (1/1) [Help](#)

Last updated (UTC)
September 10, 2026 at 15:49:09

[Action](#)[Run](#)[Create crawler](#)

View and manage all available crawlers.

< 1 >



☒	Name	State	Schedule	Last run	Last run times...	Log	Tab
☒	pulsecharge-data-lake-crawler	Ready		Succeeded	September 10, 20...	View log	9 cr

Tables

A table is the metadata definition that represents your data, including its schema. A table can be used as a source or target in a job definition.

Tables (9)

Last updated (UTC)
September 10, 2026 at 15:50:15

[Delete](#)[Add tables using crawler](#)[Add table](#)

View and manage all available tables.

Choose catalog

385697366557

Choose database

pulsecharge_catalog_db

< 1 >



☐	Name	Database	Location	Classification	Deprecated	View data	Data quality	Column statis...
☐	chargers_ports_csv	pulsecharge_catalog_db	s3://pulsecharge-data-lake-crawler/pulsecharge_data/chargers_ports_csv	CSV	-	Table data	View data quality	View statistics
☐	charging_sessions_csv	pulsecharge_catalog_db	s3://pulsecharge-data-lake-crawler/pulsecharge_data/charging_sessions_csv	CSV	-	Table data	View data quality	View statistics
☐	customer_vehicles_csv	pulsecharge_catalog_db	s3://pulsecharge-data-lake-crawler/pulsecharge_data/customer_vehicles_csv	CSV	-	Table data	View data quality	View statistics
☐	customers_csv	pulsecharge_catalog_db	s3://pulsecharge-data-lake-crawler/pulsecharge_data/customers_csv	CSV	-	Table data	View data quality	View statistics
☐	ev_models_csv	pulsecharge_catalog_db	s3://pulsecharge-data-lake-crawler/pulsecharge_data/ev_models_csv	CSV	-	Table data	View data quality	View statistics
☐	maintenance_ticket_csv	pulsecharge_catalog_db	s3://pulsecharge-data-lake-crawler/pulsecharge_data/maintenance_ticket_csv	CSV	-	Table data	View data quality	View statistics
☐	payments_csv	pulsecharge_catalog_db	s3://pulsecharge-data-lake-crawler/pulsecharge_data/payments_csv	CSV	-	Table data	View data quality	View statistics
☐	stations_csv	pulsecharge_catalog_db	s3://pulsecharge-data-lake-crawler/pulsecharge_data/stations_csv	CSV	-	Table data	View data quality	View statistics
☐	tariffs_csv	pulsecharge_catalog_db	s3://pulsecharge-data-lake-crawler/pulsecharge_data/tariffs_csv	CSV	-	Table data	View data quality	View statistics

Click on any table name to open its details and verify that AWS Glue correctly detected column names and data types (such as string, bigint, or double).

Tables

stations_csv

ied Studio

itoring

bles

is

estration)

tions

New

registries

on and ETL

Description

-

Last updated

September 10, 2026 at 15:48:12

Connection

-

Advanced properties

Schema

Partitions

Indexes

Column statistics - new

Schema (8)

Edit schema as JSC

View and manage the table schema.

Filter schemas

#	Column name	Data type	Partition key	Comment
1	station_id	bigint	-	-
2	station_name	string	-	-
3	category	string	-	-
4	address	string	-	-
5	city	string	-	-
6	latitude	double	-	-
7	longitude	double	-	-
8	installation_date	string	-	-

Amazon Athena

Query 1

✕

✓ Query 1

✕

1

SELECT * FROM "pulsecharge_catalog_db"."stations" limit 10;

aws

Search

[Alt+S]

Canada (Central)

us-east-1 (us-east-1)

Amazon Athena

Query editor

charging_sessions_csv

customer_vehicles

customer_vehicles_csv

customers

customers_csv

ev_models

ev_models_csv

maintenance_tickets

Views (0)

Query results

Query statistics

Finished

Queuing time : 184 ms

Execution time : 454 ms

Data analyzed : 2.86 KB

Results (10)

Copy

Download CSV results

Search for lines

#	station_id	station_name	category	address	city	latitude	longitude
1	1	PulseCharge - Jenny Motorway	Suburban Mall	0971 Jason Locks Suite 007	Phoenix	26.094984	70.0
2	2	PulseCharge - Teresa Ports	Residential Transit	144 Harris Grove	Chicago	-83.446719	-33.1
3	3	PulseCharge - Hannah Valleys	Residential Transit	096 Conner Street	Vancouver	-77.304471	-26.1
4	4	PulseCharge - Carly Motorway	Suburban Mall	255 Malik Route Suite 629	Seattle	17.851782	19.7
5	5	PulseCharge - Saunders Drive	Residential Transit	865 Miller Glen Apt. 557	Vancouver	31.002313	109.1
6	6	PulseCharge - Pamela Junctions	Suburban Mall	582 Tiffany Turnpike	Houston	-34.752552	32.7
7	7	PulseCharge - Gilmore Parkway	Urban Hub	190 Michael Shores	Toronto	-45.788171	-58.1
8	8	PulseCharge - Burns Islands	Urban Hub	686 Barbara Summit Suite 137	Toronto	-48.204283	-78.1
9	9	PulseCharge - Tyler Cliffs	Highway Corridor	64381 Andrew Burgs Suite 132	Seattle	-70.850847	-14.4
10	10	PulseCharge - Tapia Manors	Residential Transit	609 Cooper Landing Apt. 469	Toronto	68.293916	76.8

With storage and ad-hoc querying in place, we can now move to

Data Transformation & Pipeline Automation (Glue ETL).

In this phase, we will set up a Glue Studio job to transform your raw CSVs into compressed, columnar Parquet format and save them in a transformed-data/ S3 folder for production analytics.

Step-by-Step Setup: CSV to Parquet Pipeline

1.Create a New Visual ETL Job:

In the AWS Glue Console, go to ETL jobs in the left navigation menu.

Select Visual ETL.

Under Create job, leave the default visual options and click Create.

Click the Job details tab at the top.

Set Name to: **pulsecharge-csv-to-parquet-job**.

Set IAM Role to: **PulseChargeGlueRole**.

Scroll to Advanced properties, find Maximum concurrency, and set it to 3.

2.Configure the S3 Source Node (Raw CSV):

Click the Visual tab at the top left to return to the canvas.

Click the + (Add node) button or select Source → Amazon S3.

Click the new Amazon S3 Source node on the canvas.

In the right panel under Data source properties - S3

S3 source type: Select S3 location.

S3 URL: Enter **s3://pulsecharge-data-lake-102026/raw-data/charging_sessions/**

Data format: Select CSV.

Delimiter: Select Comma (,).

Check the box for First line contains column headers.

3.Configure the S3 Target Node (Transformed Parquet):

Click the + (Add node) button or select Target → Amazon S3.

Click the new Amazon S3 Target node on the canvas.

In the right panel under Data target properties - S3:

Node parents: Select your Amazon S3 Source node.

Format: Select Parquet.

Compression Type: Select Snappy.

S3 Target Location: Enter **s3://pulsecharge-data-lake-102026/transformed-data/charging_sessions/**

Expand Advanced options and set Target folder action to Delete existing keys in the target directory.

4. Save and Execute:

Click the blue Save button in the upper-right corner.

Click Run.

How to Verify

Click the Runs tab at the top of the canvas.

Wait 1–2 minutes for the status to show Succeeded.

Navigate to **s3://pulsecharge-data-lake-102026/transformed-data/charging_sessions/** in the S3 console to see your generated .parquet files.

Job runs (1/1) [View details](#) [Stop job run](#) [Troubleshoot with AI](#) [Table View](#) [Card View](#)

Run status	Retries	Start time (Local)	End time (Local)	Duration	Capacity (DPUs)	Worker type	Glue version
✔ Succeeded	0	09/10/2026 14:00:26	09/10/2026 14:02:19	1 m 28 s	10 DPUs	G.1X	5.1

Run details | Input arguments (11) | Logs | Run insights | Metrics | Troubleshooting analysis | Spark UI

Job name	Start time (Local)	DPU Hours	Glue version
pulsecharge-csv-to-parquet-job	09/10/2026 14:00:26	0.246944	5.1
Last modified on (Local)	Id	End time (Local)	Worker type
09/10/2026 14:02:19	jr_0901329604fbd034fcc7443f3e66ca5ae1f4c5989bf835ed251082f8dff277b6	09/10/2026 14:02:19	G.1X
Log group name	Run status	Start-up time	Max capacity
/aws-glue/jobs	✔ Succeeded	24 seconds	10 DPUs
Number of workers	Retry attempt number	Execution time	Execution class
10		1 minute 28 seconds	Standard
Initial run			

Now that the Glue ETL job has succeeded, our raw CSV data has been converted into compressed Parquet files sitting in your transformed-data/ folder.

The final phase is to run a Glue Crawler so the AWS Glue Data Catalog auto-discovers your Parquet schema, allowing you to run fast **SQL queries in Amazon Athena**.

AWS Glue Studio [Q help](#)

Get Started [Q help](#)



Author in a visual interface focused on data flow.

[Visual ETL](#)



Author using an interactive code notebook.

[Notebook](#)



Author code with a script editor.

[Script editor](#)



Use SageMaker Unified Studio Notebooks, an immersive, serverless workspace.

[Set up](#)

► Example jobs [Q help](#)

[Create example job](#)

Your jobs (1) [Q help](#)



Actions ▼

Run job

< 1 > ⚙

☐	Job name	Type	Created by	Last modified	AWS Glue version	Action
☐	pulsecharge-csv-to-parquet-job	Glue ETL	Visual	9/10/2026, 1:59:32 PM	5.1	Upgrade with AI

Crawlers

A crawler connects to a data store, progresses through a prioritized list of classifiers to determine the schema for your data, and then creates metadata tables in your data catalog.

Crawlers (2) [Q help](#)

Last updated (UTC)
September 10, 2026 at 18:16:31



Action ▼

Run

[Create crawler](#)

View and manage all available crawlers.

< 1 > ⚙

☐	Name	State	Schedule	Last run	Last run times...	Log	Ta
☐	pulsecharge-data-lake-crawler	✓ Ready		✓ Succeeded	September 10, 20...	View log	10
☐	pulsecharge-parquet-crawler	✓ Ready		-	-	-	-

≡ [Amazon Athena](#) > Query editor

[Editor](#)

[Recent queries](#)

[Recorded queries](#)

[Query parameters](#)

Data



Data source

[AwsDataCatalog](#)

Catalog

[None](#)

Database

[pulsecharge_catalog_db](#)

✓ Query 1 :

1 SELECT * FROM "pulsecharge_catalog_db"."charging_sessions" LIMIT 10;

Editor

Recent queries

Recorded queries

Query parameters

Data

↺

↻

Data source

AwsDataCatalog

Catalog

None

Database

pulsecharge_catalog_db

Query 1

```
1 SELECT
2     customer_id,
3     COUNT(session_id) AS total_sessions,
4     ROUND(SUM(energy_kwh), 2) AS customer_total_kwh
5 FROM "pulsecharge_catalog_db"."charging_sessions"
6 GROUP BY customer_id
7 HAVING SUM(energy_kwh) > 50
8 ORDER BY customer_total_kwh DESC;
```

Query results		Query statistics	
Finished		Queueing time : 94 ms	Execution time : 408 ms
		Data analyzed : 84.97 KB	
Results (202)		Copy Download CSV results	
Q Search for lines		< 1 ... > ⚙	
#	customer_id	total_sessions	customer_total_kwh
1	42	18	964.33
2	227	19	892.27
3	62	16	840.17
4	292	17	816.68
5	233	14	772.58
6	272	15	753.18
7	26	15	719.0
8	299	14	666.78
9	59	12	662.55
10	128	15	657.29

Plain-Language Method Explanation: This query groups all charging logs by individual customer IDs and uses the HAVING clause to filter out low-usage accounts. It isolates only high-volume customers whose total lifetime consumption exceeds 50 kWh.

Business Meaning & Value: Identifies high-value EV drivers and commercial fleet operators generating the highest usage on the network. Marketing teams can target these accounts with loyalty rewards or corporate subscription packages to maximize customer retention.

Query 3: Session Energy Categorization (Classification Logic)SQL

- Analysis Technique: Dynamic Classification & Binning

Editor Recent queries Recorded queries Query parameters

Data

Data source

AwsDataCatalog

Catalog

None

Database

pulsecharge_catalog_db

Tables and views

Create

Query 1

```
1 SELECT
2     session_id,
3     customer_id,
4     energy_kwh,
5     CASE
6         WHEN energy_kwh >= 40 THEN 'High Consumption'
7         WHEN energy_kwh >= 15 THEN 'Medium Consumption'
8         ELSE 'Low Consumption'
9     END AS load_category
10 FROM "pulsecharge_catalog_db"."charging_sessions"
11 LIMIT 25;
```

Query results

Query statistics

Finished

Queueing time : 58 ms

Execution time : 401 ms

Data analyzed : 84.97 KB

Results (25)

Copy

Download CSV results

Search for lines

< 1 > ⚙

#	session_id	customer_id	energy_kwh	load_category
1	1	132	70.88	High Consumption
2	2	216	82.63	High Consumption
3	3	6	26.82	Medium Consumption
4	4	140	73.39	High Consumption
5	5	121	36.33	Medium Consumption
6	6	299	26.13	Medium Consumption
7	7	113	54.72	High Consumption
8	8	31	43.23	High Consumption
9	9	121	42.24	High Consumption
10	10	101	25.33	Medium Consumption
11	11	123	23.18	Medium Consumption

#	session_id	customer_id	energy_kwh	load_category
11	11	123	23.18	Medium Consumption
12	12	26	79.09	High Consumption
13	13	145	11.56	Low Consumption
14	14	4	28.25	Medium Consumption
15	15	123	15.73	Medium Consumption
16	16	97	20.98	Medium Consumption
17	17	79	47.06	High Consumption
18	18	297	32.71	Medium Consumption
19	19	68	14.27	Low Consumption
20	20	252	51.57	High Consumption
21	21	128	43.79	High Consumption
22	22	292	55.74	High Consumption
23	23	42	69.19	High Consumption
24	24	269	64.28	High Consumption
25	25	240	19.67	Medium Consumption

Plain-Language Method Explanation: This query uses conditional logic (CASE WHEN) to assign each charging session to a load category ("High Consumption", "Medium Consumption", or "Low Consumption") based on pre-defined energy consumption thresholds.

Business Meaning & Value: Categorizes raw session data into actionable operational tiers. Grid operations teams can monitor the ratio of "High Consumption" sessions to manage electrical transformer loads and mitigate power surge risks during peak operational hours.

Query 4: Customer Usage Tier Ranking

- Analysis Technique: Tiered Segmentation via Window Functions

Editor Recent queries Recorded queries Query parameters

Data

Data source

AwsDataCatalog

Catalog

None

Database

pulsecharge_catalog_db

Query 1

```
1 SELECT
2   customer_id,
3   ROUND(SUM(energy_kwh), 2) AS total_kwh,
4   NTILE(3) OVER (ORDER BY SUM(energy_kwh) DESC) AS usage_tier
5 FROM "pulsecharge_catalog_db"."charging_sessions"
6 GROUP BY customer_id
7 ORDER BY total_kwh DESC;
```

Query results Query statistics

Finished Queueing time : 64 ms Execution time : 398 ms Data analyzed : 84.97 KB

Results (213) Copy Download CSV results

Search for lines

#	customer_id	total_kwh	third_use
1	42	964.33	1
2	227	892.27	1
3	62	840.17	1
4	292	816.68	1
5	233	772.58	1
6	272	753.18	1
7	26	719.0	1
8	299	666.78	1
9	59	662.55	1
10	128	657.29	1
11	277	642.08	1

#	customer_id	total_kwh	third_use
86	129	263.46	2
87	274	259.74	2
88	54	255.97	2
89	87	255.87	2
90	257	253.98	2
91	109	250.68	2
92	246	250.35	2
93	145	248.61	2
94	99	236.92	2
95	38	236.48	2
96	169	236.04	2
97	275	234.59	2
98	157	231.57	2
99	266	231.51	2
100	167	228.59	2

Plain-Language Method Explanation: This query calculates total energy consumed per customer and applies the NTILE(3) window function to rank and divide the entire customer base into three equal statistical tiers (Tier 1 = Top 33% users, Tier 2 = Middle 33%, Tier 3 = Bottom 33%).

Business Meaning & Value: Provides an objective breakdown of customer behavior without relying on arbitrary cutoff values. The business can design tiered pricing strategies—offering peak discounts to Tier 3 users to encourage usage, while offering Tier 1 users premium access rights

We successfully built an end-to-end cloud pipeline: from raw CSV storage in Amazon S3, through serverless transformation using AWS Glue Visual ETL into Snappy-compressed Parquet, down to automated cataloging and optimized querying in Amazon Athena.

Phase 8: Dashboard & Final Submission Checklist

Dashboard Requirements (Power BI / QuickSight / Excel)

- 5 Visuals:
 1. Line Chart: Hourly Energy Demand (kWh) by Station Cluster.
 2. Bar Chart: Revenue by Charger Type (DC Fast vs Level 2).
 3. Donut Chart: Charger Port Status breakdown (Available, Occupied, Offline).
 4. Scatter Plot: K-Means Station Clusters (Session Count vs. Revenue).
 5. Table: Top 10 High-Downtime Stations requiring maintenance.
- 2 Interactive Slicers: Date Range Selector, Region/Station Cluster Filter

Conclusion:

PulseCharge Data Lake & Analytics Pipeline

- The PulseCharge project successfully established a scalable, cost-effective, and automated end-to-end data engineering pipeline within the AWS cloud environment (ca-central-1).
- By implementing a modern medallion-style architecture, the project transformed raw, unstructured CSV datasets into optimized columnar storage formats, unlocking high-performance querying and actionable business insights.